

UNIT 3

Introduction to SciPy

What is SciPy?

- SciPy = Scientific Python Library.
- Built on top of NumPy.
- Provides advanced mathematical functions and algorithms.
- Used for scientific computing, engineering, and data analysis.

Why use SciPy?

- Faster than writing complex math functions manually.
- Provides ready-to-use scientific algorithms.
- Used in Machine Learning, Image Processing, Optimization, Signal Processing, Statistics, and more.
- Works with NumPy arrays.

Installing SciPy

```
pip install scipy
```

Importing SciPy

```
import scipy
```

```
from scipy import linalg, stats    # Import specific modules
```

5. Major Sub-packages of SciPy

Sub-package	Purpose
-------------	---------

scipy.linalg	Linear algebra (matrices, eigenvalues)
--------------	--

scipy.fftpack	Fast Fourier Transform (FFT)
---------------	------------------------------

scipy.optimize Optimization (min/max problems, curve fitting)

scipy.integrate Integration and solving differential equations

scipy.interpolate Interpolation of data points

scipy.signal Signal processing (filters, convolution)

scipy.sparse Sparse matrices

scipy.stats Probability and statistics

scipy.ndimage Image processing

6. Examples of SciPy Usage

a) Linear Algebra

```
import numpy as np
```

```
from scipy import linalg
```

```
A = np.array([[1, 2], [3, 4]])
```

```
det = linalg.det(A)      # Determinant
```

```
print("Determinant:", det)
```

b) Integration

```
from scipy import integrate
```

```
f = lambda x: x**2
```

```
res, err = integrate.quad(f, 0, 1)      #  $\int x^2 dx$  from 0 to 1
```

```
print("Integral:", res)
```

c) Statistics

```
from scipy import stats
```

```
data = [1, 2, 2, 3, 4, 4, 4, 5]
```

```
mean = np.mean(data)
```

```
mode = stats.mode(data, keepdims=True)
print("Mean:", mean)
print("Mode:", mode.mode[0])

d) Optimization

from scipy.optimize import minimize

f = lambda x: x**2 + 2*x + 1

result = minimize(f, 0) # minimize function starting at x=0

print("Minimum at:", result.x)
```

Introduction to NLTK

1. What is NLTK?

- NLTK = Natural Language Toolkit.
- A popular Python library for Natural Language Processing (NLP).
- Provides easy-to-use tools for working with human language data (text).
- Useful for text preprocessing, tokenization, stemming, lemmatization, POS tagging, parsing, and sentiment analysis.

2. Why use NLTK?

- Simplifies text processing.
- Provides a large set of corpora (datasets) and lexical resources (like WordNet).
- Supports machine learning models for NLP tasks.
- Widely used for education, research, and prototyping NLP projects.

3. Installing NLTK

```
pip install nltk
```

Then, download NLTK resources (run in Python shell once):

```
import nltk

nltk.download('all') # Or download specific packages
```

4. Importing NLTK

```
import nltk

from nltk.tokenize import word_tokenize, sent_tokenize
```

5. Key Features of NLTK

Feature	Description
Tokenization	Splitting text into words or sentences
Stemming	Reducing words to root form (e.g., "running" → "run")
Lemmatization	Smarter word reduction using vocabulary (e.g., "better" → "good")
Stopword Removal	Removing common words like the, is, and
POS Tagging	Identifying parts of speech (noun, verb, etc.)
Parsing	Analyzing sentence structure
Sentiment Analysis	Detecting emotions (positive/negative)
Corpora Access	Pre-loaded datasets for NLP research

6. Examples

a) Tokenization

```
from nltk.tokenize import word_tokenize, sent_tokenize

text = "NLTK is a powerful library. It is used for text processing."

print(sent_tokenize(text)) # Sentence tokenization

print(word_tokenize(text)) # Word tokenization
```

b) Stemming

```
from nltk.stem import PorterStemmer

ps = PorterStemmer()

words = ["running", "runner", "ran", "runs"]

print([ps.stem(w) for w in words])
```

c) Lemmatization

```
from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')

lemmatizer = WordNetLemmatizer()

print(lemmatizer.lemmatize("running", "v"))    # → run
print(lemmatizer.lemmatize("better", "a"))      # → good
```

d) Stopwords

```
from nltk.corpus import stopwords

nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

words = ["this", "is", "a", "sample", "sentence"]

filtered = [w for w in words if w not in stop_words]

print(filtered)    # ['sample', 'sentence']
```